Jira - Jira DC Hashing Approach

Jira DC, what is the best approach to keep some data hashed in memory in SIL or groovy script, to be able to call it from any ticket anytime

In **Jira Data Center**, if you want **hashed data cached in memory and accessible from scripts executed on any issue**, the optimal approach depends on the scripting framework you are using and the **cluster characteristics** of DC. SIL and Groovy behave differently, and the key design questions are:

- **Scope**: per script, JVM node, or cluster-wide
- **Persistence**: transient cache vs stored state
- **Concurrency / thread safety**
- **Cluster replication**

Below are the **recommended architectural options**, ordered from most robust to simplest.

---

1. Use the Jira Cache API (Best for Groovy / ScriptRunner)

For Groovy scripts, the **native Jira cache layer** is the most correct solution.

Jira provides **cluster-aware caches** through:

- `com.atlassian.cache.CacheManager`
- `CacheFactory`

These caches are designed for **Jira Data Center**, meaning they support:

- replication
- eviction policies
- thread safety

Example Groovy Cache

```
import com.atlassian.jira.component.ComponentAccessor
import com.atlassian.cache.CacheManager
import com.atlassian.cache.CacheSettingsBuilder
import java.util.concurrent.TimeUnit

def cacheManager = ComponentAccessor.getComponent(CacheManager)

def cache = cacheManager.getCache(
    "custom.hashed.data.cache",
    { key -> null },  // loader
    new CacheSettingsBuilder()
        .expireAfterWrite(10, TimeUnit.MINUTES)
        .replicateViaInvalidation()
        .build()
)

// Store hashed data
cache.put("user_email_hash", "a84fe34d...")

// Retrieve
def value = cache.get("user_email_hash")
```

Advantages

- cluster-safe
- no memory leaks
- supports TTL
- thread-safe
- accessible from **any Groovy script**

When to use

✔️ ScriptRunner environments
✔️ enterprise Jira DC
✔️ frequently accessed computed data

---

2. Static Singleton Map (Node-local cache)

A simpler but **node-local** method is a **static map inside a class**.

Example

```
class HashCache {

    static Map<String,String> cache = new ConcurrentHashMap<>()

}
```

Usage in scripts:

```
HashCache.cache.put("key", "hashedvalue")

def value = HashCache.cache.get("key")
```

Characteristics

Feature	Result
Cluster-safe	❌ no
Node memory	✔️
Fast	✔️
Persistence	❌

Each **Jira node has its own cache**.

When to use

✔️ temporary calculations
✔️ non-critical cached data

---

3. Plugin Shared Cache (Best enterprise solution)

For large implementations you should create a **small Jira plugin** exposing a shared service:

```
HashStorageService
```

Internally it uses:

```
CacheManager
```

Then Groovy or SIL calls the service.

Architecture:

```
Groovy / SIL
      ↓
Plugin REST / Service
      ↓
Jira Cache API
```

Advantages:

- cluster-wide
- reusable
- secure
- versioned

---

4. SIL Global Variables (Limited)

SIL scripts support **global variables**, but they are **not designed for shared runtime caches**.

Example:

```
global string HASH_CACHE[];
```

Problems:

- not guaranteed across nodes
- limited lifecycle
- not efficient for large maps

Use only for **small script state**.

---

5. AO Table or DB Table (Persistent hash storage)

If hashes must survive restarts:

Store them in

- **Active Objects table**
- or **custom DB table**

Then cache them in memory using method #1.

Pattern:

```
DB (source of truth)
   ↓
Jira Cache
   ↓
Scripts
```

Recommended Architecture

For **Jira DC production systems**:

```
Jira DB / AO
      |
CacheManager
      |
Cluster Replicated Cache
      |
Groovy / ScriptRunner scripts
      |
Any Jira issue
```

This gives:

- **O(1) lookup**
- cluster consistency
- eviction control
- minimal DB load

Important DC Considerations

1. Nodes do not share memory

Static maps work **per node only**.

2. Script reload clears static maps

ScriptRunner reloads may clear them.

3. Cache size limits

Always configure eviction policies.

Example Use Case (Personal Data Hashing)

Since you mentioned **detecting personal information in Jira** (from your earlier SQL audit work), a good pattern is:

```
Raw value → SHA256 → store in cache
```

Cache key:

```
email:john@company.com
```

Value:

```
3b5d5c3712955042212316173ccf37be...
```

Then any issue script can instantly check:

```
is this value already seen?
```

If you want, I can also show

- a **cluster-safe distributed hash registry for Jira DC**
- a **Groovy + SIL hybrid cache service**
- a **JQL profiler cache architecture (used in large enterprises)**

These patterns are often used in **100k+ issue Jira installations**.

Give me architecture of cache API and principle of work as well as how persistent is this approach to save data

In **Jira Data Center**, the caching layer is provided by the Atlassian **Cache API**, exposed through `com.atlassian.cache.CacheManager`. It is designed for **high-performance, cluster-aware in-memory storage** with optional replication and eviction policies. Understanding how it works requires looking at its **architecture, lifecycle, and persistence model**.

---

1. Cache API Architecture

At a high level, the Jira DC cache system sits between application code and the underlying data source.

```
Application Scripts / Plugins
        │
        │  get / put
        ▼
   CacheManager
        │
        ▼
  Named Cache Instance
        │
  ┌──────────────────────┬──────────────────────┐
  │ Local Node           │ Cluster Nodes        │
  │ Memory Cache         │ Replication          │
  └──────────────────────┴──────────────────────┘
        │
        ▼
 Optional Source of Truth
 (Database / AO / External)
```

Key components

1. CacheManager

- central factory for caches
- ensures consistent configuration
- creates **named caches**

2. Cache

- key-value store
- thread-safe
- supports loaders and eviction

3. CacheSettings
defines behavior such as:

- TTL
- replication strategy
- max entries

---

2. Internal Implementation

In **Jira Data Center**, Atlassian Cache API is backed by **Ehcache** with cluster replication.

Conceptually:

```
CacheManager
   │
   ├── Node-local memory store (Ehcache)
   │
   ├── Replication channel
   │        │
   │        ▼
   │   Other DC nodes
   │
   └── Eviction / expiry manager
```

Each node has its **own in-memory cache**.

Cluster coordination happens via **replication events**.

---

3. Cache Replication Modes

Cache settings determine how nodes synchronize.

1. Local cache (default)

```
Node A cache
Node B cache
Node C cache
```

No synchronization.

Pros

- fastest
- no network traffic

Cons

- inconsistent between nodes

---

2. Replicate via invalidation (recommended)

```
Node A updates cache
        │
        ▼
broadcast invalidation
        │
        ▼
Nodes B/C drop local value
```

Each node reloads the value when requested.

Pros

- consistent
- low network overhead
- safe for distributed systems

---

3. Replicate via copy

```
Node A writes value
      │
      ▼
Nodes B/C receive same value
```

Pros

- immediate consistency

Cons

- heavier network usage

---

4. Principle of Work (Lifecycle)

Typical workflow:

Step 1 — Cache creation

```
CacheManager.getCache("my-cache")
```

Jira registers a **named cache instance**.

Caches are **singleton per node**.

Step 2 — Data lookup

```
value = cache.get(key)
```

Flow:

```
request
  │
  ▼
check node memory
  │
  ├── hit → return value
  │
  └── miss
        │
        ▼
     loader or DB
        │
        ▼
    store in cache
```

Step 3 — Cache update

```
cache.put(key, value)
```

Flow:

```
update local cache
      │
      ▼
replication event (optional)
      │
      ▼
other nodes update / invalidate
```

Step 4 — Eviction

Eviction happens when:

- TTL expired
- max entries exceeded

- explicit invalidation
- node restart

---

5. Persistence Characteristics

The **Atlassian Cache API is NOT persistent storage**.

Important properties:

Property	Behavior
Stored location	JVM memory
Survives node restart	❌ No
Survives Jira restart	❌ No
Survives cluster restart	❌ No
Survives script reload	✔ Usually
Shared across nodes	✔ Optional

What happens during restart

```
Jira node restart
      |
      ▼
All cache memory cleared
      |
      ▼
Caches rebuilt lazily
```

The cache must be **repopulated from source**.

---

6. Recommended Persistence Pattern

Because the cache is ephemeral, production systems use a **two-layer architecture**.

```
Source of Truth
(DB / AO table)
      |
      ▼
CacheManager
Cluster Cache
      |
      ▼
Groovy / SIL
Jira Scripts
```

Cache = **fast lookup layer**

Database = **persistent storage**

---

7. Typical Performance

Cache operations are extremely fast:

Operation	Latency
Cache read	~50-200 ns
Concurrent map lookup	~100 ns
DB query	1-10 ms
REST call	10-100 ms

This means cache lookup can be **10,000x faster than DB access**.

---

8. Example for Hash Storage

If you want to store hashed values (for example email hashes):

```
key: email_hash:<email>
value: sha256 hash
```

Access flow:

```
Issue Script
    │
    ▼
Cache lookup
    │
 ├── hit → return hash
 │
 └── miss
        │
        ▼
calculate hash
        │
        ▼
store in cache
```

---

9. Memory Considerations

Memory consumption roughly equals:

```
(key size + value size + overhead) × entries
```

Example:

```
hash key 40 bytes
value 64 bytes
overhead ~80 bytes

≈ 180 bytes per entry
```

1 million entries ≈ **180 MB per node**

---

10. When Cache API Is the Right Tool

Use it when data is:

✅ frequently accessed
✅ computationally expensive
✅ safe to recompute
✅ read much more often than written

Avoid it when data must be:

❌ permanent
❌ auditable
❌ strongly transactional

---

11. For Your Use Case (Cross-issue hashed lookup)

Best architecture would be:

```
            Hash Registry
                  │
                  ▼
          Persistent DB Table
                  │
                  ▼
          Cluster Cache
                  │
                  ▼
         Issue Groovy/SIL Script
```

This gives:

- **instant lookup**
- **cluster safety**
- **restart recovery**

If you want, I can also show a **production-grade cache architecture used in large Jira DC installations (100k+ users)** that includes:

- **multi-level caching (L1 + L2)**
- **Bloom filters for hash detection**
- **automatic warm-up on node startup**
- **cluster-safe eviction strategy**

This pattern works extremely well for **JQL profiling, PII detection, and rule engines in Jira**.

Compare saving data in cache and in CF

In **Jira Data Center**, storing data in the **Cache API** and storing data in a **Custom Field (CF)** serve fundamentally different architectural purposes. One is an **in-memory performance layer**, while the other is **persistent application data stored in the Jira database**. Below is a technical comparison focused on how they behave in **large Jira DC environments**, such as those you are working with when building audit and analysis tooling.

---

1. Core Architectural Difference

Cache API

```
Script / Plugin
      |
      ▼
  CacheManager
      |
      ▼
 Node Memory (per JVM)
      |
      ▼
 Optional cluster replication
```

- stored **only in RAM**
- managed by **Ehcache**
- **not persisted**

Custom Field

```
Script / Plugin
      |
      ▼
Custom Field API
      |
      ▼
Jira Issue Layer
      |
      ▼
Jira Database Tables
```

Typical tables involved:

- `customfieldvalue`
- `jiraissue`
- `changegroup`
- `changeitem`
- `fieldconfigscheme`

Custom field values become part of the **issue data model**.

---

2. Persistence

Property	Cache API	Custom Field
Stored in	JVM memory	Jira DB
Survives Jira restart	❌	✅
Survives node restart	❌	✅
Survives cluster restart	❌	✅
Survives reindex	N/A	✅
Backup via DB	❌	✅

Conclusion

Cache = **ephemeral**

Custom Field = **permanent**

---

3. Performance

Typical operation latency:

Operation	Cache	Custom Field
Read	~0.1 µs	2–10 ms
Write	~0.2 µs	5–20 ms
Cluster propagation	~1–10 ms	DB commit

Cache is roughly **10,000x faster** than reading from issue fields.

Reason:

- CF access involves
  - DB query
  - issue object construction
  - permission checks
  - indexing layer

---

4. Impact on Jira System

Cache

Impact is minimal.

- does **not affect indexing**
- does **not affect issue size**
- does **not affect workflows**
- does **not create change history**

Custom Field

Every value update triggers:

```
Update issue
     │
     ▼
DB write
     │
     ▼
Lucene reindex
     │
     ▼
Change history entry
```

Consequences:

- indexing load
- database growth
- issue update events
- audit history

On large systems this can become expensive.

---

5. Scalability

Cache

Memory usage example:

```
Entry size ≈ 180 bytes
1M entries ≈ 180 MB per node
```

Scaling behavior:

- scales **linearly per node**
- very fast lookup
- limited by heap size

Custom Fields

Data growth example:

```
1M issues × field value
```

Stored in:

```
customfieldvalue table
```

Effects:

- DB growth
- index size increase
- slower reindex operations

Large installations often have **hundreds of millions of CF rows**.

---

6. Querying Capability

Cache

You cannot query caches using:

- JQL
- SQL
- REST

Access only through:

```
cache.get(key)
```

Custom Fields

Full ecosystem support:

- JQL
- REST API
- SQL queries
- dashboards
- automation rules

Example:

```
cf[12345] = "hashvalue"
```

---

7. Data Visibility

Aspect	Cache	Custom Field
Visible in UI	❌	✅
Visible via REST	❌	✅
Visible in SQL	❌	✅
Searchable in JQL	❌	✅

Cache is **internal system state**, not business data.

---

8. Cluster Behavior

Cache

Two modes:

1. **Node local**

```
Node A cache
Node B cache
```

2. **Replicated**

```
Node A writes
      |
      ▼
Invalidation event
      |
      ▼
Nodes B/C reload
```

Custom Field

Cluster-safe by design.

```
Node A
   |
   ▼
Database write
   |
   ▼
Other nodes read updated value
```

---

9. Typical Use Cases

Cache

Best for:

- computed values
- temporary hash registries
- rule evaluation data
- JQL profiler results

- expensive lookup tables

Example:

```
email → SHA256
```

Custom Field

Best for:

- business data
- reporting
- workflow state
- audit information
- JQL searchable values

Example:

```
PII detection result
Compliance classification
Risk score
```

---

10. Operational Cost

Cache

Operational overhead:

- minimal
- memory consumption only
- no index impact

Custom Field

Operational impact:

- DB size growth
- reindex cost
- issue history growth
- API load

Large DC instances sometimes remove CFs for this reason.

---

11. Data Lifecycle

Cache lifecycle

```
script computes value
      |
      ▼
store in cache
      |
      ▼
value expires
      |
      ▼
recomputed when needed
```

Custom field lifecycle

```
workflow/script sets value
      |
      ▼
stored permanently
      |
      ▼
visible in issue history
```

---

12. Best Architecture for Your Use Case

Since you mentioned **detecting personal data in Jira issues**, the optimal pattern is usually:

```
Persistent Layer
        │
        ▼
DB / AO table
        │
        ▼
Cache Layer
        │
        ▼
ScriptRunner / SIL
        │
        ▼
    Issues
```

Where:

- **cache** provides fast lookup
- **DB** ensures restart persistence

Custom fields should only store **results you want visible or searchable**.

---

13. Quick Decision Matrix

Requirement	Use Cache	Use CF
Ultra-fast lookup	✔	
Data must survive restart		✔
Visible to users		✔
Searchable via JQL		✔
Temporary calculations	✔	
Large hash registry	✔	
Audit/reporting		✔

✔ **Short rule**

- **Cache = computation layer**
- **Custom field = business data**

If you'd like, I can also show a **very powerful pattern used in enterprise Jira DC**: a **three-level data model** combining **Cache + AO tables + custom fields** that allows:

- instant lookup
- restart persistence
- JQL reporting
- minimal indexing overhead.

Compare with storing data in the file

In **Jira Data Center**, storing data in **Cache**, **Custom Fields**, or **Files on disk** represents three very different architectural models: **memory cache**, **database-backed application data**, and **filesystem persistence**. Each has different performance, durability, cluster behavior, and operational implications.

Below is a structured comparison relevant for **Groovy / ScriptRunner / SIL automation** and large Jira DC environments.

---

1. Architectural Models

Cache API (Memory Layer)

```
Script / Plugin
      |
      ▼
 CacheManager
      |
      ▼
 JVM Memory (Ehcache)
      |
      ▼
 Optional cluster replication
```

Purpose: **fast runtime lookup**

Custom Field (Application Data Layer)

```
Script / Workflow
      |
      ▼
 Custom Field API
      |
      ▼
 Jira Issue Model
      |
      ▼
 Jira Database
```

Purpose: **business data attached to issues**

File Storage (Filesystem Layer)

```
Script
   |
   ▼
File API (Groovy/Java IO)
   |
   ▼
Local Disk
   |
   ▼
Shared filesystem (NFS in DC)
```

Purpose: **external persistent data**

---

2. Persistence

Property	Cache	Custom Field	File
Stored in	RAM	DB	Disk
Survives Jira restart	❌	✔	✔
Survives node restart	❌	✔	✔
Survives cluster restart	❌	✔	✔
Included in DB backup	❌	✔	❌
Included in filesystem backup	❌	❌	✔

Files are persistent but **outside Jira data model**.

---

3. Performance

Typical read latency:

Operation	Time
Cache lookup	~0.1 µs
File read (local)	~0.5–5 ms
File read (NFS)	~5–20 ms
Custom field read	~2–10 ms

Cache is **orders of magnitude faster**.

File reads are often **slower than DB reads in clustered setups** due to network filesystem latency.

---

4. Cluster Behavior (Jira Data Center)

Cache

Each node has its own memory.

```
Node A cache
Node B cache
Node C cache
```

Optional replication via invalidation.

Custom Fields

Fully cluster-safe through the database.

```
Node A
  |
  ▼
DB write
  |
  ▼
Node B / C read updated value
```

File Storage

Depends on configuration.

Option 1 — local disk

```
Node A writes file
Node B cannot see it
```

This **breaks in DC**.

Option 2 — shared filesystem (NFS)

```
Node A writes
      |
      ▼
Shared storage
      |
      ▼
Nodes B/C read
```

But introduces:

- network latency
- locking complexity
- failure modes

---

5. Concurrency and Safety

Cache

Thread-safe.

Ehcache manages:

- locking
- concurrency
- eviction

Custom Fields

Safe via Jira transaction layer.

Includes:

- DB transactions
- indexing
- audit history

Files

You must manage:

- file locks
- concurrent writes
- corruption risk

Example issues:

```
two scripts writing same file
partial write
read during write
```

Requires manual synchronization.

---

6. Querying and Visibility

Capability	Cache	CF	File
Search via JQL	❌	✅	❌
REST API	❌	✅	❌
SQL queries	❌	✅	❌
User visibility	❌	✅	❌

Files and cache are **internal storage only**.

---

7. Operational Impact

Cache

Impact:

- JVM memory usage
- no DB load
- no indexing

Very lightweight.

Custom Fields

Impact:

- DB growth
- index size increase
- issue history growth
- slower reindex

Large Jira DC systems sometimes have **hundreds of millions of CF rows**.

Files

Impact:

- filesystem growth
- backup complexity
- possible NFS contention

Large filesystems can affect cluster IO performance.

---

8. Typical Use Cases

Cache

Good for:

- lookup tables
- hash registries
- rule engines
- computed values
- JQL performance profiling

Example:

```
email → SHA256 hash
```

Custom Fields

Good for:

- business metadata
- workflow state

- reporting
- JQL filtering

Example:

```
PII classification level
```

Files

Good for:

- large datasets
- logs
- exports
- external configuration

Example:

```
hash dictionary
pattern library
large ML model
```

---

9. Scalability

Cache

Memory limited.

Example:

```
200 bytes per entry
1M entries ≈ 200 MB per node
```

But extremely fast.

Custom Fields

DB limited.

Example:

```
1M issues × field value
```

Stored in `customfieldvalue`.

Large installations can exceed **100M rows**.

Files

Disk limited.

Example:

```
hash dictionary file
10M hashes ≈ 600 MB
```

Filesystem handles large data better than DB or memory.

---

10. Data Lifecycle

Cache

```
compute → store → expire → recompute
```

Ephemeral.

Custom Field

```
workflow update → DB → index → permanent
```

Permanent application data.

File

```
write → persistent until manually removed
```

External persistence.

---

11. Best Architecture for Hash Registry

For your use case (cross-issue data hashing / PII detection), the most scalable pattern is:

```
Persistent Registry
(File or DB)
|
▼
Jira Cache Layer
|
▼
Issue Scripts
```

Startup flow:

```
node start
|
▼
load file into cache
|
▼
fast runtime lookup
```

This combines:

- **persistent storage**
- **fast memory lookup**

---

12. Quick Comparison Summary

Feature	Cache	Custom Field	File
Speed	★★★★★	★★	★
Persistence	❌	✔️	✔️
Cluster-safe	✔️	✔️	⚠️ depends
JQL searchable	❌	✔️	❌
Operational overhead	low	high	medium
Data size capacity	medium	large	very large

✔️ **Short rule**

- **Cache → runtime speed**
- **Custom Field → Jira business data**
- **File → large persistent external data**

If you'd like, I can also show a **very powerful enterprise pattern used in large Jira DC clusters**:

Distributed Hash Registry

which combines:

- filesystem persistence
- Cache API memory index
- Bloom filters for instant detection

and can handle **tens of millions of hashes with sub-millisecond lookup inside Jira scripts**.